교내 GPU 관리 서버 개발 프로젝트 · 아키텍처 기술 검토

교내망 검토 보고서: 접근 계층 및 NAS 연결 설계

작성자: 김상현

핵심 요약

- **접근 계층 구성 제안:** 교내 GPU 노드가 AWS EC2 중앙 서버로 SSH 역방향 터널(Reverse Tunnel)을 먼저 여는 프록시 구조를 제안합니다. (EC2 ↔ GPU1 구간 평균 RTT 4.029ms, 처리량 약 93Mbps 확인)
- **오버레이(NetBird) 방식 검토 결과:** 캠퍼스 NAT(Port Restricted Cone) 환경에서 P2P 연결은 성립하나, 연속 전송 시 높은 UDP 패킷 손실(다운로드 52.13%)로 수신 지연 현상이 나타나고 다운로드 처리량이 9.23Mbps 에 그쳐 현 단계의 접근 계층으로는 적합하지 않은 것으로 판단됩니다.
- **외부 대역폭 상한과 전송률 제어:** GPU1·GPU2 동시 전송 시에도 합산 대역폭이 약 91~93Mbps 수준으로 확인되어, 외부 출구 게이트웨이에 100Mbps 링크 제한이 있는 것으로 보입니다. 또한 대용량 다운로드 중 평균 RTT 가 11.817ms(p95 20.300ms)에서 183.550ms(p95 184.800~207.900ms)로 증가하므로, SFTP 연결당 전송률 제한(Rate Limit) 적용을 권장합니다.
- **NAS 연결 방식 제안:** GPU 노드에서 교내 NAS 를 SMB(TCP 445)로 직접 마운트하여 학습 데이터 입출력이 EC2 나 SSH 터널을 거치지 않도록 분리하는 구성을 제안합니다. (I/O 성능은 추후 도입할 NAS 환경에서 측정 예정)

1. 조사 배경 및 측정 환경

백엔드 및 시스템 계층 설계에 앞서, 교내 방화벽과 NAT 환경에서 외부 사용자 접속 경로(접근 계층)와 스토리지(NAS) 연결 방식을 도출하기 위해 네트워크 실측을 진행했습니다.

- **사전 환경 및 경로 비교 조사:** GPU1·GPU2 의 캠퍼스 NAT 특성 확인, Bastion 서버 경유 접속과 오버레이(NetBird) P2P 접속의 성능 비교
- **역방향 터널 및 스토리지 심화 조사:** EC2 기반 SSH 역방향 터널의 처리량·안정성 검증, 교내 NAS 마운트 확인, A6000 서버의 네트워크 연결 경로 확인

1.1. 조사 대상 장비

장비명	네트워크 대역	역할 및 비고

GPU1, GPU2	사설 (192.168.0.*)	성능 측정의 주 대상인 기존 교내 GPU 서버
A6000 서버	공인 (210.94.*.*)	A6000 2 장을 장착한 타 연구실 소유 GPU 서버
제 2 연구실 PC	사설 (192.168.0.67)	A6000 서버 및 NAS 관리 화면 접속 시 거치는 점프 호스트
EC2 중앙 서버	AWS 공인	외부 사용자 접속을 교내 GPU 노드로 중계하는 중앙 서버
Bastion 서버	공인 / 내부 접점	외부에서 GPU1·GPU2 및 제 2 연구실 PC 를 관리할 때 거치는 접속 서버
교내 NAS	공인 (210.94.*.170)	교내 설치 Synology NAS (SMB 경로 및 마운트 확인용)
클라이언트 노트북	외부망	외부 사용자 환경 모사 및 구간 성능 측정 장비

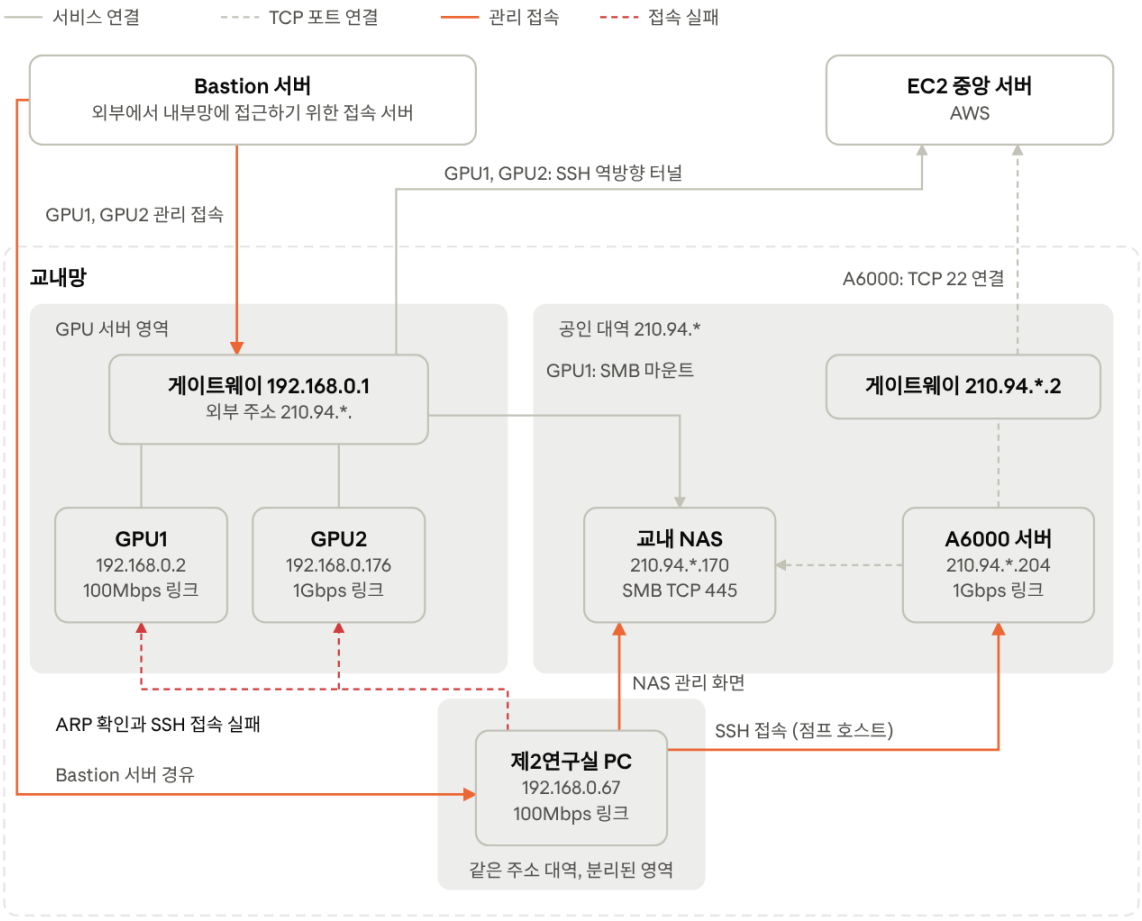
1.2. 측정 항목 및 방법

SSH 터널은 TCP 연결만 전달하므로, 왕복 지연 시간(RTT)은 ICMP ping 대신 TCP 요청·응답 시간을 측정하는 bounceback 방식을 사용했습니다.

확인 항목	측정 도구 및 방법
경로 및 포트 접근	ip route get, nc (TCP 포트 연결 확인)
인터페이스 링크 속도	ethtool (NIC 지원 최대 속도 및 현재 협상 속도 확인)
TCP 처리량 (Throughput)	iperf2 (60 초씩 3 회 연속 전송, 수신 측 측정값 기준)
왕복 지연 (RTT)	iperf2 TCP bounceback (100B 요청을 1 초 간격으로 전송, p95 는 100µs 히스토그램 근삿값)
실사용 부하 및 공존성	SFTP 파일 전송 중 SSH 명령 실행(300 회) 및 신규 로그인(48 회) 성공 여부·소요 시간
터널 유지 안정성	6 시간 동안 30 초 주기로 SSH 배너 수신 확인 (타임아웃 5 초)
NAS 파일 접근	SMB 마운트(TCP 445) 및 파일 생성 권한 확인

2. 교내 네트워크 구성 및 링크 특성

교내 장비는 사설 대역(GPU1·GPU2, 제 2 연구실 PC)과 공인 대역(A6000, 교내 NAS)에 나뉘어 연결되어 있으며, 세 GPU 서버 모두 EC2(TCP 22) 및 교내 NAS(TCP 445)로의 아웃바운드 TCP 연결이 가능한 상태입니다.



[그림 1] 교내 네트워크 구성도

2.1. 장비별 연결 속도

장비	인터페이스	지원 최대 속도	현재 연결 속도	비고 및 추정 원인
GPU1	enp4s0	1Gbps (전이중)	100Mbps	스위치 포트는 1Gbps 를 지원하나 케이블·커넥터 문제로 감속된 것으로 보임
GPU2	enp2s0	1Gbps (전이중)	1Gbps	1Gbps 정상 연결 (단, 외부 전송 시 상위 게이트웨이 대역폭 영향)
제 2 연구실 PC	enp4s0	1Gbps (전이중)	100Mbps	연결된 상위 스위치 포트가 100Mbps 까지만 지원
A6000 서버	enp1s0f1	10Gbps (전이중)	1Gbps	상대 스위치 장비 또는 케이블이 1Gbps 까지만 지원하는 것으로 보임

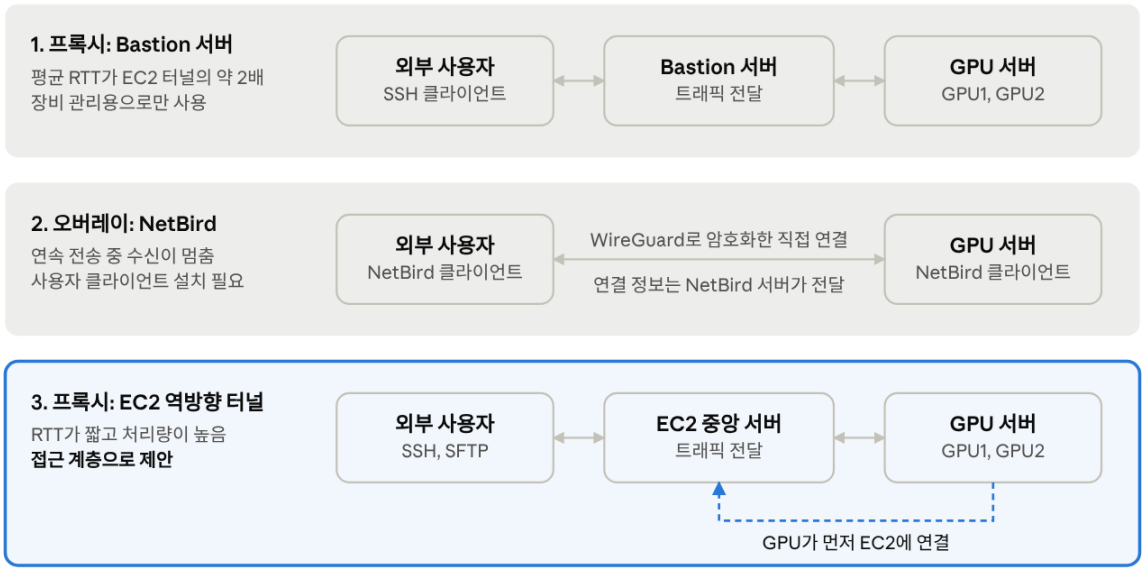
2.2. 내부망 분리 구조와 라우팅 경로

- **GPU1 ↔ GPU2 내부 통신:** 게이트웨이를 거치지 않고 동일 내부망에서 직접 통신하며, 양방향 평균 수신 처리량은 94.10Mbps(GPU1 의 100Mbps 링크 한계), ping 평균 RTT 는 0.261~0.387ms(손실 0%)로 확인되었습니다.
- **제 2 연구실 PC 의 영역 분리:** 주소 대역(192.168.0.67)과 기본 게이트웨이 주소(192.168.0.1)는 두 GPU 와 같으나, 두 GPU 의 사설 주소에 대한 ARP 확인 및 SSH 접속이 이루어지지 않아 논리적으로 분리된 네트워크 영역(VLAN 등)에 속한 것으로 판단됩니다.
- **관리 경로와 데이터 경로:** 관리 접속 시에는 Bastion 서버나 제 2 연구실 PC(점프 호스트)를 거치지만, 각 서버가 EC2 및 NAS 로 여는 데이터 연결은 아래 표의 다음 홉(Next Hop)으로 직접 나갑니다. 따라서 제 2 연구실 PC 의 100Mbps 링크가 A6000 서버의 데이터 전송 속도를 제한하지 않습니다.

출발지	목적지 (포트)	다음 홉 (Next Hop)	확인 범위
GPU1, GPU2	EC2 (TCP 22)	192.168.0.1	SSH 인증, 역방향 터널 수립, 양방향 데이터 전송
A6000 서버	EC2 (TCP 22)	210.94.*.2	TCP 포트 연결 확인
GPU1	교내 NAS (TCP 445)	192.168.0.1	SMB 인증 및 마운트 확인
GPU2	교내 NAS (TCP 445)	192.168.0.1	TCP 포트 연결 확인
A6000 서버	교내 NAS (TCP 445)	직접 통신 (동일 대역)	TCP 포트 연결 확인

3. 접근 계층 검토: 프록시와 오버레이 비교

교내망 방화벽 정책상 외부에서 내부로의 인바운드 연결이 제한되므로, 내부 GPU 서버가 공인 주소를 가진 외부 서버로 먼저 연결을 여는 구조가 필요합니다. 중계 서버가 트래픽을 전달하는 프록시(Proxy) 방식과 양 끝단이 직접 통신하는 오버레이(Overlay P2P) 방식을 비교한 결과, 응답 지연이 짧고 연속 전송 안정성이 확인된 EC2 SSH 역방향 터널(프록시) 방식을 접근 계층으로 제안합니다.



[그림 2] 접근 방식별 데이터 경로

3.1. 캠퍼스 NAT 특성 및 오버레이(NetBird) 검토

- **캠퍼스 NAT 동작 특성:** GPU1·GPU2 에서 tailscale netcheck 및 turnutils_natdiscovery 로 확인한 결과, 외부 STUN 서버와의 UDP 송신이 허용되고 목적지가 달라도 외부 주소·포트가 유지되나, 응답하는 외부 주소나 포트가 바뀌면 수신을 차단하는 Port Restricted Cone NAT 특성을 보였습니다.
- **종단 간(노트북 ↔ GPU2) RTT 및 TCP 처리량:** 사전 환경 조사에서 동일한 외부 노트북과 GPU2 구간의 RTT 를 비교한 결과, NetBird P2P 연결은 기존 중계 서버(Bastion) 포트포워딩 방식보다 지연 시간은 짧았으나 TCP 연속 전송 시 수신이 일시 중단되는 현상이 나타났고 평균 처리량도 업로드 34.20Mbps, 다운로드 9.23Mbps 에 머물렀습니다.

접속 방식 (노트북 ↔ GPU2)	응답 수	평균 RTT	최대 RTT
기존 중계 서버(Bastion)를 통한 SSH 포트포워딩	90 개	22.429 ms	657.904 ms
NetBird P2P	90 개	9.064 ms	19.967 ms

- **방향별 UDP 전송 시험 결과:** NetBird 오버레이 주소로 송신 부하를 낮춰 방향별 UDP 성능을 확인한 결과, 업로드는 수신 처리량 52.5Mbps 와 손실률 0.035%를 기록한 반면 다운로드는 약 21Mbps(-b 20M)의 송신 설정에서도 수신 처리량이 9.27Mbps 에 그치고 손실률은 52.13%로 높게 나타났습니다. 다운로드 방향의 수신 중단과 패킷 손실이 두드러지고 사용자 측 클라이언트 설치 부담이 있는 점을 고려해 접근 계층 후보에서는 제외하는 것이 적절하다고 판단됩니다.

방향 및 송신 설정 (iperf2)	수신 처리량	패킷 손실률
노트북 → GPU2 (업로드, -b 50M / 약 52.43Mbps)	52.50 Mbps	0.035%
GPU2 → 노트북 (다운로드, -b 20M / 약 20.97Mbps)	9.27 Mbps	52.13%

3.2. 프록시 경로 비교: Bastion 경유와 EC2 역방향 터널

측정 대상	접속 경로	평균 RTT	최대 RTT	EC2 → GPU 처리량	GPU → EC2 처리량
GPU1	SSH 역방향 터널	4.029 ms	33.375 ms	93.17 Mbps	93.80 Mbps
GPU1	Bastion 서버 경유	8.686 ms	18.933 ms	85.40 Mbps	85.27 Mbps
GPU2	SSH 역방향 터널	4.205 ms	43.837 ms	수신 로그 미확보	93.67 Mbps
GPU2	Bastion 서버 경유	9.348 ms	22.285 ms	수신 로그 미확보	85.03 Mbps

- **지연 시간 및 처리량 비교:** SSH 역방향 터널은 기존 중계 서버(Bastion)를 경유할 때보다 중간 경유지가 줄어들어 평균 RTT 가 약 절반 수준(4.029~4.205ms)으로 짧았고 수신 처리량도 약 8Mbps 높았습니다. 다만 60 초×3 회 측정 중 관측된 최대 RTT 는 GPU1(33.375ms)과 GPU2(43.837ms) 모두 역방향 터널에서 더 크게 나타났습니다.
- **EC2 → GPU2 송신 측 처리량:** 위 표는 수신 측 처리량을 기준으로 비교하므로 GPU2 의 EC2 → GPU 구간은 '수신 로그 미확보'로 표기했습니다. 동일 시험에서 확보된 EC2 송신 측 기록 기준으로는 SSH 역방향 터널이 평균 82.57Mbps(회차별 82.4 / 79.6 / 85.7Mbps), Bastion 서버 경유가 평균 75.50Mbps(회차별 81.7 / 68.7 / 76.1Mbps)로 역방향 터널이 약 7Mbps 높았습니다.
- **단일 터널 내 병렬 연결(1 개 vs 4 개) 비교:** GPU2 에서 동일한 외부 SSH 연결 하나를 공유하는 iperf 연결을 1 개에서 4 개로 늘려 측정한 결과, GPU2 → EC2 방향의 합산 수신 처리량은 93.67Mbps 에서 93.30Mbps 로 증가하지 않았습니다.

전송 방향 (GPU2)	연결 1 개 평균	연결 4 개 합계 평균	집계 위치
EC2 → GPU2	82.57 Mbps	94.20 Mbps	EC2 송신 측 (수신 로그 미확보)
GPU2 → EC2	93.67 Mbps	93.30 Mbps	EC2 수신 측

4. 프록시 경로(SSH 역방향 터널)의 실사용 성능

4.1. 두 GPU 노드의 동시 전송과 외부 대역폭 상한

각 GPU 를 단독으로 측정한 값과 두 GPU 에서 동시에 전송한 값(60 초×3 회 평균)을 비교했습니다.

전송 방향	GPU1 단독	GPU2 단독	동시 전송 (GPU1)	동시 전송 (GPU2)	동시 전송 합계
EC2 → GPU	88.30 Mbps	89.20 Mbps	34.37 Mbps	56.40 Mbps	90.77 Mbps
GPU → EC2	92.90 Mbps	92.80 Mbps	14.10 Mbps	78.63 Mbps	92.73 Mbps

GPU2 는 NIC 가 1Gbps 로 연결되어 있음에도 단독 처리량이 약 93Mbps 수준이었고, 두 GPU 를 동시에 사용해도 합산 처리량은 약 91~93Mbps 로 유지되었습니다. 두 GPU 가 동일한 기본 게이트웨이(192.168.0.1)와 외부 공인 IP(210.94.*.33)를 공유하는 점을 고려하면, 외부 출구 구간에 100Mbps 링크 제한이 존재하는 것으로 보입니다. 따라서 해당 구간이 유지되는 환경에서는 동일 게이트웨이 하단에 GPU 노드를 추가하더라도 외부 전송 총량은 늘어나지 않을 것으로 예상됩니다.

4.2. 대용량 파일 전송 중 응답 지연 변화 및 TCP 재전송

노트북 ↔ EC2 ↔ GPU1 경로에서 대용량 파일 전송 부하(70 초 유지)와 대화형 패킷의 RTT 측정(부하 시작 5 초 뒤부터 60 초간 응답 60 개 집계)을 같은 터널에서 동시에 수행했습니다. 아래 표의 업로드·다운로드 범위는 3 회 측정 각각의 최솟값과 최댓값입니다.

측정 조건	평균 RTT	RTT 표준편차	p95 RTT	최대 RTT	수신 처리량
파일 전송 없음	11.817 ms	4.187 ms	20.300 ms	31.399 ms	-
업로드 중	62.319 ms	11.418 ~ 13.155 ms	82.700 ~ 96.300 ms	108.226 ~ 124.101 ms	89.77 Mbps
다운로드 중	183.550 ms	14.397 ~ 25.196 ms	184.800 ~ 207.900 ms	221.264 ~ 361.659 ms	92.13 Mbps

대용량 전송으로 100Mbps 출구 대역폭이 포화되면서 같은 경로의 평균 RTT 가 업로드 중 약 5 배(p95 82.700~96.300ms), 다운로드 중 약 15 배(p95 184.800~207.900ms)로 증가했습니다. 원활한 SSH 터미널 응답성을 유지하기 위해 구현 계획에 파일 전송률 제한(Rate Limit)을 반영하는 방안을 제안합니다.

또한 동일 시험에서 실제 TCP 송신 구간별로 수집한 TCP 재전송 바이트 비율(bytes_retrans 증가량 / bytes_sent 증가량 × 100, 유휴 시 모든 구간 0%)은 다음과 같습니다.

부하 조건 및 실제 TCP 송신 구간	1 회 측정	2 회 측정	3 회 측정
업로드 중 (EC2 → GPU1 구간)	0.212049% (1,171 회)	0.193299% (1,050 회)	0.176727% (982 회)
다운로드 중 (GPU1 → EC2 구간)	0.000000% (0 회)	0.000000% (0 회)	0.007632% (43 회)
다운로드 중 (EC2 → 노트북 구간)	0.000349% (3 회)	0.000117% (1 회)	0.000232% (2 회)

4.3. SFTP 전송률 제한 시 SSH 동시 사용 및 장기 유지 관측

- **SSH 와 SFTP 동시 사용 시험:** SFTP 전송률을 연결당 16,000Kbit/s(약 16Mbps)로 제한하고 256MiB 파일 송수신(7 회)과 최대 4 개의 SSH 세션을 함께 사용한 조건에서, 기존 연결의 SSH 명령 300 회(조건별 60 회)와 신규 로그인 48 회(조건별 12 회)가 모두 정상 완료되었습니다. 명령 1 회당 평균 완료 시간은 유휴 시 36.233ms, 파일 전송 병행 시 39.617~52.117ms 였고, 파일 송수신과 SSH 를 함께 사용한 세 회차에서 신규 로그인 평균 시간은 271.833~280.667ms, SSH 명령 완료 시간의 p95 는 49.000~72.000ms 로 확인되었습니다.

측정 조건	신규 로그인 평균	SSH 명령 p95	SSH 명령 최대
파일 전송 없음 (유휴 기준)	303.000 ms	43.000 ms	45.000 ms
업로드 + SSH 병행	해당 없음	47.000 ms	52.000 ms
업로드·다운로드 + SSH + 신규 로그인 (1 회)	280.667 ms	49.000 ms	146.000 ms
업로드·다운로드 + SSH + 신규 로그인 (2 회)	280.583 ms	49.000 ms	77.000 ms
업로드·다운로드 + SSH + 신규 로그인 (3 회)	271.833 ms	72.000 ms	133.000 ms

- **터널 6 시간 연속 관측:** 30 초 주기로 신규 연결의 SSH 배너 수신(5 초 이내)을 확인한 결과, 총 720 회 중 719 회 성공(99.861%)했습니다. 응답이 지연된 1 회 역시 터널 프로세스 자체의 단절 기록은 없었으며, 터널을 인위적으로 종료한 시험에서는 재연결 대기 시간(5 초)을 포함해 약 7 초 만에 복구되었습니다.

5. NAS 연결 방식

NAS 는 각 GPU 노드에서 SMB(TCP 445)로 직접 마운트하여, 저장소 데이터 트래픽이 외부 EC2 나 SSH 터널을 거치지 않도록 구성하는 방식을 제안합니다.

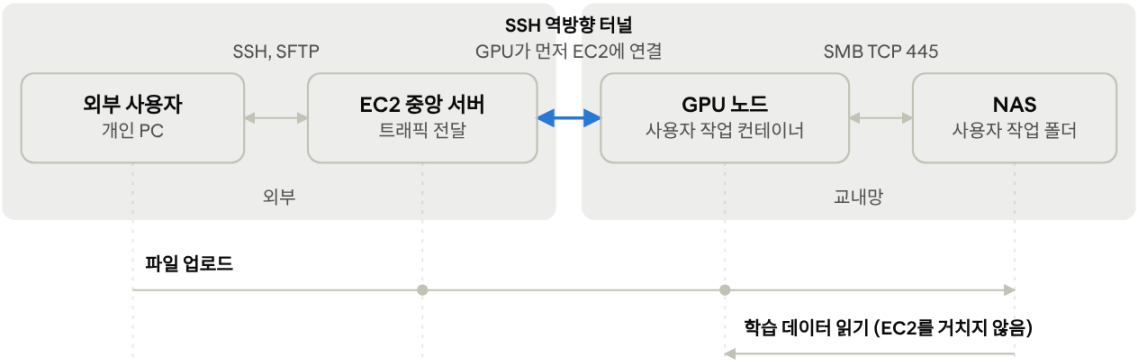
확인 항목	GPU1 (사설 대역)	GPU2 (사설 대역)	A6000 (공인 대역)
TCP 445 포트 연결	연결 확인 (192.168.0.1 경유)	연결 확인 (192.168.0.1 경유)	연결 확인 (동일 대역)
SMB 인증 및 마운트	마운트 확인	미수행	추후 연동 시 확인
입출력(I/O) 성능 측정	쓰기 권한 부재로 미수행	미수행	미수행

현재 교내 NAS 는 테스트 계정에 쓰기 권한이 부여되지 않아 마운트 동작까지만 확인했습니다. GPU 노드와 NAS 사이의 처리량·지연 시간, 대용량 순차 읽기 및 다수 소형 파일 읽기 성능은 추후 도입할 NAS 환경에서 평가할 예정입니다.

6. 제안하는 네트워크 아키텍처 및 MVP 구현 방안

6.1. 전체 서비스 연결 구조

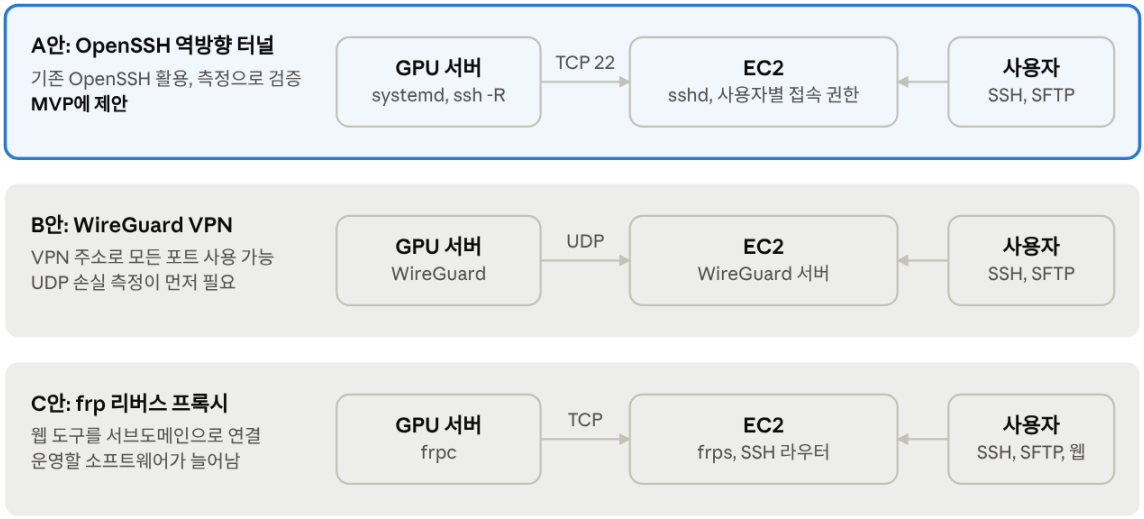
MVP 단계에서는 GPU 노드가 EC2 중앙 서버에 SSH 역방향 터널을 먼저 연결하고, 외부 사용자는 EC2 를 경유해 자신의 작업 컨테이너에 접속하는 구성을 제안합니다. 사용자가 업로드하는 파일은 사용자 → EC2 → GPU → NAS 경로로 저장되며, 컨테이너 내부 학습 코드가 데이터를 읽을 때는 교내망 내부 경로(GPU ↔ NAS)로 직접 읽어 EC2 대역폭을 소모하지 않습니다.



[그림 3] 제안하는 서비스 구성

6.2. GPU 와 EC2 연결 기술 비교

GPU 노드를 EC2 에 연결하는 세 가지 기술 안을 비교한 결과, MVP 에는 터널 연결에 기존 OpenSSH 를 활용할 수 있고 실측 검증을 마친 A 안(OpenSSH 역방향 터널)을 제안합니다.



[그림 4] GPU 와 EC2 연결 기술 세 가지 안

구분	A 안: OpenSSH 역방향 터널 (MVP 제안)	B 안: WireGuard VPN	C 안: frp 리버스 프록시
GPU 측 실행 요소	systemd 서비스 (ssh -N -R)	WireGuard 클라이언트	frpc 데몬
EC2 측 실행 요소	OpenSSH sshd, 사용자별 접속 권한 관리	WireGuard 서버, SSH 라우터	frps, SSH 라우터
연결 프로토콜	TCP 22	UDP	TCP
GPU 추가 설치	터널 연결에는 기존 OpenSSH 활용	WireGuard 패키지 설치 필요	frpc 실행 파일 배포 필요
검증 상태	GPU1, GPU2 에서 측정 및 검증	미검증	미검증
고려할 위험 요인	단일 터널 공유 시 대용량 전송이 타 사용자 응답 지연 유발 가능	캠퍼스 외부 방향 UDP 전송 시 패킷 손실 발생 이력 존재	관리 대상 소프트웨어 및 인증 설정 요소 증가

6.3. A 안(OpenSSH 역방향 터널) 구현 절차 예상

EC2 와 교내 GPU 서버 사이에서 생성과 종료 요청은 HTTPS 로 전달하고, 사용자 접속과 파일 전송은 SSH 역방향 터널로 처리합니다.

1. **생성과 종료 요청 전달**: EC2 의 Spring Boot 백엔드가 사용자의 생성과 종료 요청을 받습니다. 교내 관리 프로그램은 HTTPS 로 EC2 에 주기적으로 접속해 처리할 요청을 가져오고, Kubernetes API 에 전달합니다. 실제 처리 결과를 EC2 에 보고하며, 같은 요청을 중복 실행하지 않도록 관리합니다.
2. **작업 환경 생성과 접속**: Kubernetes 와 HAMi 로 작업용 컨테이너를 만들고 GPU 자원을 배정합니다. GPU 노드에서 EC2 로 OpenSSH 역방향 터널을 연결해 사용자가 EC2 를 거쳐 작업 환경에 접속하도록 합니다. 터널은 인스턴스별 Kubernetes Service 에 연결해 컨테이너 주소 변경에 대응합니다.
3. **본인 작업 환경에만 접속 허용**: 웹터미널은 백엔드가 웹 로그인과 소유권을 확인한 뒤 SSH 접속을 대신합니다. 개인 노트북은 등록한 SSH 키와 ProxyJump 로 접속하며, EC2 의 AuthorizedKeysCommand 를 통해 본인 작업 환경으로만 연결하도록 제한합니다. 접속 대상의 호스트키는 웹터미널 서버와 노트북에서 각각 검증합니다.
4. **연결 복구와 사용 종료**: OpenSSH 와 systemd 로 끊어진 터널을 복구합니다. 네트워크 연결만 끊겼을 때는 실행 중인 학습을 유지합니다. 사용자가 인스턴스를 종료하면 터널 재연결을 중단하고, 기존 접속과 컨테이너를 정리한 뒤 GPU 자원을 반환합니다.
5. **파일 전송 속도 조절**: 대역폭 제한을 지원하는 Kubernetes CNI 로 작업 환경별 전송 속도를 제한합니다. 제한값은 공유 회선의 용량과 대용량 파일 전송 중 SSH 응답 속도를 확인해 정합니다.
6. **파일 저장과 보존**: NAS 접근 권한을 확보한 뒤 SMB CSI 로 GPU 노드에 연결하고, 사용자별 저장 공간을 PV 와 PVC 로 제공합니다. 인스턴스를 종료해도 파일이 남도록 저장 공간 삭제를 별도로 처리하고, PV 에는 Retain 정책을 적용합니다.

7. 설계 반영 제안 및 잔여 확인 사항

7.1. 백엔드·시스템 설계 및 기획서 반영 제안

도출된 네트워크 특성	측정 근거	기획서 및 시스템 설계 반영 제안
외부 전송 용량이 노드 수에 비례하지 않음	두 GPU 동시 전송 합계 약 91~93Mbps	프록시 대역폭 상한을 중계 서버(EC2) 사양이 아닌 교내 외부 출구 링크(약 93Mbps) 기준으로 명시하고, 동시 전송 수 및 전송량 배분 설계에 반영
파일 전송 중 대화형 응답	다운로드 중 평균 RTT	SFTP 연결당 전송률 제한(Rate Limit)을 적용하고

지연 증가	183.550ms(p95 최대 207.900ms)로 증가	컨테이너별 터널을 분리하여 SSH 접속 품질 유지
일시적인 터널 지연·단절 가능성	6 시간 중 1 회 응답 지연, 강제 종료 시 약 7 초 후 복구	터널 상태를 상시 감시하되, 일시적인 통신 단절만으로 실행 중인 학습 컨테이너를 종료하지 않도록 상태 관리 로직 구성
노드 간 직접 통신을 가정하기 어려움	제 2 연구실 PC 영역 분리 및 A6000 대역 상이	다중 노드 관리 시 노드 간 직접 통신을 전제하지 않고, 각 노드의 에이전트가 중앙 서버(EC2)로 연결하는 구조 채택 추가 GPU 서버를 같은 클러스터에 편입하기 전, Kubernetes 제어 통신과 노드 간 Pod 통신을 확인합니다.
스토리지 연결 상태 확인 필요	NAS 마운트 및 권한 오류 시 파일 유실 우려	인스턴스 생성 단계에서 저장소(NAS) 마운트와 접근 확인이 완료된 경우에만 접속 가능 상태로 전환

7.2. 추가 확인 필요 항목

- **신규 NAS 입출력 성능 평가:** 추후 도입할 NAS 에서 GPU 노드와의 처리량·지연 시간, 대용량 파일 순차 읽기 및 다수 소형 파일 읽기 성능, 다중 사용자 동시 접근 성능을 측정합니다.
- **GPU1 물리 링크 점검:** 100Mbps 로 협상된 GPU1 의 랜 케이블 및 커넥터를 교체하여 1Gbps 연결이 정상 복구되는지 확인합니다. (현재는 GPU1 ↔ NAS 간 내부 전송도 100Mbps 로 제한됨)
- **컨테이너 격리 접속 및 데이터 영속성 검증:** 개인 노트북(ProxyJump)과 웹터미널에서 EC2 를 거쳐 사용자 본인 컨테이너에만 SSH·SFTP 로 접속되는지, 다른 사용자의 작업 환경으로는 연결이 거부되는지, 인스턴스 종료 후에도 NAS 저장 파일이 정상 유지되는지 확인합니다.
- **장기 변동성 및 교내 무선 환경 비교:** 하루 이상의 장기 처리량·지연·패킷 손실 추이를 관측하고, 교내 유선 및 무선 접속 환경에서의 차이를 비교합니다.
- **관리 부서 정책 및 타 연구실 장비 협의:** 교내 네트워크 관리 부서에 역방향 터널 운영 관련 정책을 확인하고, 타 연구실 소유인 A6000 서버의 서비스 연동(인증, 터널, NAS 마운트) 전 TF 장 협의를 진행합니다.